MAGNATE ACADEMY

Technical Round 1 – Agentic AI

Examination Questions & Answers

Contents

Part 1 — Python Coding and Theory

Part 2 — Agentic AI Theory

Part 3 — Practical AI Agent Project

Prepared as a beginner-friendly reference with code examples and explanations.

PART 1 — PYTHON | 20 MARKS

A. Python Coding | 15 Marks

1. Count the Frequency of Each Character

Use a dictionary to count how many times each character appears. For each character, increase its count if it is already in the dictionary; otherwise, start its count at 1.

```
text = "programming"
frequency = {}

for char in text:
    if char in frequency:
        frequency[char] += 1
    else:
        frequency[char] = 1

print(frequency)
# Output: {'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

2. Find the Second-Largest Distinct Number (without sort/sorted)

Keep track of the largest and second-largest distinct values while going through the list once. Repeated copies of the largest number are ignored.

```
numbers = [10, 5, 8, 20, 20, 15]
largest = None
second_largest = None

for number in numbers:
    if largest is None or number > largest:
        if number != largest:
            second_largest = largest
        largest = number
    elif number != largest and (
        second_largest is None or number > second_largest
    ):
        second_largest = number

if second_largest is not None:
    print(second_largest)
else:
    print("There is no second-largest distinct number")

# Output: 15
```

3. Remove Duplicate Elements Without Changing Order (without set)

Create an empty list and add an item only when it has not appeared in the result list yet.

```
numbers = [1, 2, 3, 2, 4, 1, 5]
unique_numbers = []

for number in numbers:
    if number not in unique_numbers:
        unique_numbers.append(number)

print(unique_numbers)
# Output: [1, 2, 3, 4, 5]
```

4. Two Sum

Use two loops to check every pair of different indices. When the two values add up to the target, print their indices. This simple approach is easy to follow.

```
nums = [2, 7, 11, 15]
target = 9
found = False
```

```
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == target:
            print([i, j])
            found = True
            break
    if found:
        break

# Output: [0, 1]
```

5. Valid Palindrome

Convert the string to lowercase and keep only letters and numbers. The cleaned string is a palindrome if it is equal to its reverse.

```
text = "A man, a plan, a canal: Panama"
cleaned = ""

for char in text:
    if char.isalnum():
        cleaned += char.lower()

if cleaned == cleaned[::-1]:
    print(True)
else:
    print(False)

# Output: True
```

B. Python Theory | 5 Marks

1. Difference Between a List and a Tuple

A **list** is an ordered, mutable collection, so its items can be changed after creation. It uses square brackets. A **tuple** is ordered but immutable, so its items cannot be changed after creation. It uses parentheses.

```
my_list = [10, 20, 30]
my_list[0] = 99
print(my_list)   # [99, 20, 30]

my_tuple = (10, 20, 30)
# my_tuple[0] = 99  # Raises TypeError
```

2. Mutable vs Immutable Data Types

Mutable objects can be changed after they are created; immutable objects cannot be changed in place. Examples of mutable types are **list** and **dictionary**. Examples of immutable types are **int** and **string** (also tuple, when its elements themselves are immutable).

3. Exception Handling: try, except, finally

Exception handling prevents a program from stopping unexpectedly when a runtime error occurs. The **try** block contains code that might fail, **except** handles a matching error, and **finally** runs whether an error occurs or not.

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except (ValueError, ZeroDivisionError):
    print("Please enter a valid non-zero number")
finally:
    print("Program finished")
```

4. List Comprehension

A list comprehension is a short way to create a new list from an iterable, optionally applying a condition.

```
squares = [number * number for number in range(1, 6)]
print(squares)
# Output: [1, 4, 9, 16, 25]
```

```
even_numbers = [n for n in range(1, 11) if n % 2 == 0]
```

5. Lambda Function vs Normal Function

A **lambda** is a small anonymous function written in one expression. A normal function is defined with **def** and can contain multiple statements and a descriptive name.

```
square = lambda x: x * x
print(square(5))  # 25

def square_normal(x):
    return x * x

print(square_normal(5))  # 25
```

PART 2 — AGENTIC AI | 10 MARKS

1. Advantages and Limitations of Reactive Agents

Advantages: Simple to build, fast to respond, needs little memory, and works well in predictable environments. **Limitations:** Reacts only to the current input, has little or no memory of past events, cannot plan far ahead, and may perform poorly when the environment changes or a situation requires reasoning.

2. Transformer Architecture and Its Importance in Modern LLMs

A Transformer is a neural-network architecture that uses attention mechanisms to understand relationships between tokens in a sequence. Self-attention helps the model weigh which words or tokens are important to one another. Transformers can process tokens in parallel during training and capture long-range context, making them a foundation of modern large language models (LLMs).

3. Prompt Engineering and Why It Matters for AI Agents

Prompt engineering means writing clear instructions and providing useful context, examples, constraints, and output formats for an AI model. It matters for agents because it helps define the task, guide tool selection and usage, reduce irrelevant responses, and make outputs more consistent.

4. System Prompt vs User Prompt

A **system prompt** sets the model's overall role, behavior, and rules (for example, “You are a helpful maths tutor”). A **user prompt** is the request or question submitted by the user (for example, “Explain fractions”). System instructions generally establish the behavior within which user requests are answered.

5. When to Choose a Multi-Agent Architecture

Choose multiple agents when a task can be divided into distinct specialities or parallel workstreams—for example, one agent researches, another analyzes data, and another writes a report. This can improve modularity and allow specialized roles. A single agent is usually simpler, cheaper, and easier to maintain for small or straightforward tasks.

6. Role, Goal, and Backstory in CrewAI

Role: The agent’s job or identity, such as “Research Analyst.” **Goal:** What the agent is expected to achieve, such as “Find reliable information about a topic.” **Backstory:** Context describing the agent’s experience or perspective, which helps shape how it approaches its work.

7. Purpose of ChatPromptTemplate

In LangChain, **ChatPromptTemplate** creates a reusable chat prompt from messages and placeholders. It lets a developer insert variables such as a user question into a consistent prompt structure, including system and user messages.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", "{question}")
])

messages = prompt.invoke({"question": "What is an AI agent?"})
```

8. Required Parameters When Creating a CrewAI Agent

The commonly required parameters are **role**, **goal**, and **backstory**. Other options, such as tools and the language model, can be supplied depending on the task and CrewAI version.

9. Default Value of max_iter in CrewAI

In commonly used CrewAI versions, the Agent parameter **max_iter** defaults to **20**. It limits the number of agent execution iterations. Defaults can vary by version, so check the installed version’s documentation if exact version-specific behavior is being tested.

10. Default Value of n in the LangChain OpenAI Completion Model

The default value of **n** is **1**, meaning one completion is requested unless another value is specified.

PART 3 — AGENTIC AI | PRACTICAL (20 MARKS)

Task: Create an AI agent using LangChain or CrewAI that accepts a user query, uses an appropriate tool, and returns a meaningful response.

Example Solution: LangChain Agent with a Calculator Tool

This example uses a calculator tool. The model can request the tool when it needs to calculate an expression. It uses a local Ollama model, so no cloud API key is needed. Tool-calling support depends on the selected model; if TinyLlama does not reliably call tools in your environment, use the deterministic calculator-only fallback described in the README.

exam.py

```
from langchain_ollama import ChatOllama
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def calculator(expression: str) -> str:
    """Calculate a mathematical expression."""
    # Use only trusted, simple arithmetic expressions.
    return str(eval(expression))

llm = ChatOllama(model="tinyllama:latest")

agent = create_agent(
    model=llm,
    tools=[calculator]
)

query = input("Enter your question: ")

result = agent.invoke({
    "messages": [{"role": "user", "content": query}]
})

print(result["messages"][-1].content)
```

Example query: “What is 25 * 4?” A successful tool call should return 100. TinyLlama may not reliably support tool calling through this agent interface; test it in the target environment. Do not use eval() with untrusted arbitrary input in a production application.

requirements.txt

```
langchain
langchain-ollama
```

Run the following commands after installing Ollama and downloading the model:

```
ollama pull tinyllama:latest
pip install -r requirements.txt
python exam.py
```

README.md — Suggested Content

Project: Calculator Tool AI Agent. This project accepts a question from the user and provides an answer using a calculator tool when a mathematical calculation is needed. It uses Python, LangChain, and a local Ollama model.

Setup: Install Ollama, ensure the Ollama service is running, pull the model with `ollama pull tinyllama:latest`, install dependencies, and run `python exam.py`.

.gitignore

```
.env
.venv/
venv/
__pycache__/
```

```
*.pyc
```

Do not upload API keys or secrets to GitHub. The example above uses a local model and does not require an API key.